



Heaps & Priority Queues

Deep Dive + Complexity + Real-World Systems

Java 8+



Session Structure

1. Heap fundamentals
2. Min Heap vs Max Heap
3. Heap operations & complexity
4. PriorityQueue in Java
5. Real-world applications
6. Heap-based problem patterns
7. Interview-level problems



What is a Heap?

A Heap is:

- A Complete Binary Tree
- Satisfies Heap Property

Types:

1. Min Heap $\rightarrow$ Parent $\leq$ Children
2. Max Heap $\rightarrow$ Parent $\geq$ Children



Complete Binary Tree

All levels filled except possibly last.

Last level filled left to right.

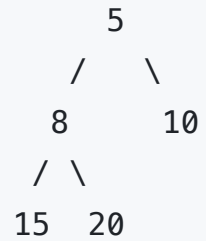
This ensures:

Height $\approx \log n$



Heap Property

Min Heap:



Root is minimum.



Array Representation

Heap stored in array:

For index i :

- Left child $\rightarrow 2i + 1$
- Right child $\rightarrow 2i + 2$
- Parent $\rightarrow (i - 1) / 2$

Space: $O(n)$



Core Operations & Complexity

Operation	Time
Insert	$O(\log n)$
Extract Min/Max	$O(\log n)$
Peek	$O(1)$
Build Heap	$O(n)$

Height = $O(\log n)$

Insert Operation (Heapify Up)

```
void heapifyUp(int[] heap, int index) {  
    while (index > 0) {  
        int parent = (index - 1) / 2;  
        if (heap[parent] <= heap[index]) break;  
  
        int temp = heap[parent];  
        heap[parent] = heap[index];  
        heap[index] = temp;  
  
        index = parent;  
    }  
}
```

Time: $O(\log n)$

Extract Min (Heapify Down)

```
void heapifyDown(int[] heap, int size, int index) {  
    while (true) {  
        int smallest = index;  
        int left = 2 * index + 1;  
        int right = 2 * index + 2;  
  
        if (left < size && heap[left] < heap[smallest])  
            smallest = left;  
        if (right < size && heap[right] < heap[smallest])  
            smallest = right;  
  
        if (smallest == index) break;  
    }  
}
```



```
int temp = heap[index];  
heap[index] = heap[smallest];  
heap[smallest] = temp;  
  
index = smallest;  
}  
}
```

Time: $O(\log n)$



Java PriorityQueue

Min Heap by default.

```
java.util.PriorityQueue<Integer> pq = new java.util.PriorityQueue<>();  
  
pq.offer(5);  
pq.offer(2);  
pq.offer(10);  
  
int min = pq.poll(); // 2
```



Max Heap in Java

```
java.util.PriorityQueue<Integer> maxHeap =  
    new java.util.PriorityQueue<>(java.util.Collections.reverseOrder());
```



Real World Use Case — Task Scheduling

Tasks with priority.

Higher priority executed first.

Used in:

- Operating systems
- Thread schedulers



Real World Use Case — Dijkstra's Algorithm

PriorityQueue stores:

Node with smallest distance.

Time complexity:

$O((V + E) \log V)$



Kth Largest Element

Time: $O(n \log k)$

Space: $O(k)$

```
static int findKthLargest(int[] nums, int k) {  
    java.util.PriorityQueue<Integer> minHeap =  
        new java.util.PriorityQueue<>();  
  
    for (int num : nums) {  
        minHeap.offer(num);  
        if (minHeap.size() > k)  
            minHeap.poll();  
    }  
    return minHeap.peek();  
}
```



Merge K Sorted Lists

Time: $O(n \log k)$

Space: $O(k)$

```
static java.util.List<Integer> mergeK(
    java.util.List<java.util.List<Integer>> lists) {

    java.util.PriorityQueue<int[]> pq =
        new java.util.PriorityQueue<>(
            (a, b) -> a[0] - b[0]);

    java.util.List<Integer> result = new java.util.ArrayList<>();

    for (int i = 0; i < lists.size(); i++)
        if (!lists.get(i).isEmpty())
            pq.offer(new int[]{lists.get(i).get(0), i, 0});
```



```
while (!pq.isEmpty()) {  
    int[] curr = pq.poll();  
    result.add(curr[0]);  
  
    int listIndex = curr[1];  
    int elementIndex = curr[2] + 1;  
  
    if (elementIndex < lists.get(listIndex).size())  
        pq.offer(new int[]{  
            lists.get(listIndex).get(elementIndex),  
            listIndex,  
            elementIndex  
        });  
}  
return result;  
}
```

Top K Frequent Elements

Time: $O(n \log k)$

```
static int[] topKFrequent(int[] nums, int k) {  
    java.util.Map<Integer, Integer> map = new java.util.HashMap<>();  
    for (int num : nums)  
        map.put(num, map.getOrDefault(num, 0) + 1);  
  
    java.util.PriorityQueue<int[]> pq =  
        new java.util.PriorityQueue<>((a, b) -> a[1] - b[1]);
```



```
for (java.util.Map.Entry<Integer, Integer> entry : map.entrySet()) {  
    pq.offer(new int[]{entry.getKey(), entry.getValue()});  
    if (pq.size() > k)  
        pq.poll();  
}  
  
int[] result = new int[k];  
for (int i = k - 1; i >= 0; i--)  
    result[i] = pq.poll()[0];  
  
return result;  
}
```



Build Heap Complexity

Building heap from array:

$O(n)$

Not $O(n \log n)$.

Reason:

Most nodes are leaves $\rightarrow$ require minimal heapify.



Common Mistakes

- Assuming heap is sorted
- Confusing BST with heap
- Forgetting complete tree property
- Misunderstanding $O(n)$ heap build



Interview Discussion Points

- Why insert is $O(\log n)$?
- Why build heap is $O(n)$?
- Difference between heap and BST?
- When to use heap over sorting?



Practice Problems

Easy:

- Kth largest element
- Last stone weight

Medium:

- Top K frequent elements
- Merge K sorted lists

Advanced:

- Median from data stream
- Design Twitter feed
- Dijkstra's shortest path



Summary

Heaps provide:

- Efficient priority handling
- $O(\log n)$ insert/remove
- Foundation of PriorityQueue
- Core to graph algorithms

Master:

- Min heap
- Max heap
- Kth element problems
- Top K pattern